COMPTIA SECURITY+ SY0-701 | DOMAIN 2 · THREATS, VULNERABILITIES, AND MITIGATIONS (22%)

Security+ 2.3 · Memory Injections

Mestre CyberSec · **Lesson 2.3 — Memory Injections** (2:39) · Assistir Vídeo no YouTube ↗ · Treino Interativo: edusouzamarques.github.io/cybersec

Executive Lesson Summary: DLL injection: força processo legítimo a carregar biblioteca maliciosa; process hollowing: substitui a memória de um processo legítimo rodando. Antivírus por assinatura não pega (o processo é 'legítimo'), só EDR behavioural.

💡 Key Exam Highlights & Critical Concepts

- DLL injection: processo carrega DLL do atacante
- Process hollowing: shellcode substitui código de processo suspendido

🎯 Exam Tips & Typical CompTIA Scenarios

- 'malicious code runs inside a legitimate process's memory' = process hollowing / DLL injection

🚨 Beyond the Video: Official Exam Concepts, Protocols & Traps (Section 2.3 — Memory Injections)

Why is this advanced coverage required for SY0-701?: Video lectures provide foundational concepts, but CompTIA tests practical engineering judgment and requires deep familiarity with the following protocols, acronyms, and operational nuances in this domain:

• DLL Injection

An attack that forces a legitimate, running process to load a malicious Dynamic-Link Library, causing attacker-controlled code to execute inside memory that appears trusted.

⚠️ ***Exam Trap:*** *Gatilho: 'processo legítimo CARREGA/é induzido a carregar uma biblioteca maliciosa' = DLL injection. O processo original continua rodando — não confunda com substituição de processo (hollowing).*

• Process Hollowing

An attack where a legitimate process is launched in a suspended state, its original code is wiped from memory, and malicious shellcode is written in its place before execution resumes, so malicious code runs as a trusted process.

⚠️ ***Exam Trap:*** *Gatilho: 'código de processo legítimo é SUBSTITUÍDO por código malicioso' ou 'processo suspenso esvaziado' = process hollowing. Distinção-chave: hollowing substitui o processo; DLL injection apenas adiciona/carrega uma biblioteca em um processo vivo.*

• Dynamic-Link Library (DLL)

A Windows shared library containing code and data that multiple programs load and call at runtime; it is the vehicle that attackers abuse to inject malicious code into legitimate processes.

⚠️ ***Exam Trap:*** *A prova pode descrever DLL como 'shared library'. A DLL em si é legítima — a maldade está no atacante forçar sua carga em um processo confiável, evadindo verificação por assinatura baseada em arquivo.*

• Shellcode

A small payload of native machine code injected directly into a process's memory and executed, typically used to replace legitimate code or to run attacker-controlled instructions.

⚠️ ***Exam Trap:*** *No process hollowing, o shellcode é o que substitui o código do processo suspenso. Se a questão menciona 'código nativo escrito nos espaços do processo', pense em shellcode/hollowing.*

• Endpoint Detection and Response (EDR)

A monitoring solution that continuously analyzes endpoint behavior, including processes, memory activity, and API calls, to detect and respond to threats like DLL injection and process hollowing that evade signature-based tools.

⚠️ ***Exam Trap:*** *Gatilho: 'detecção COMPORTAMENTAL' ou 'detecta código malicioso rodando dentro de processo legítimo' = EDR. Na prova, EDR/behavioral é a detecção correta para memory injection; antivírus por assinatura é o distrator errado.*

• Signature-based Detection

A detection method that matches files and code against known malware hashes or patterns; it misses memory injection attacks because the malicious code runs inside a legitimate, signed process's memory.

⚠️ ***Exam Trap:*** *A prova inverte o óbvio: o processo é 'legítimo', então assinatura NÃO detecta. Se a pergunta perguntar por que o antivírus falhou contra memory injection, a resposta é que a detecção por assinatura não analisa o comportamento em memória.*

• Fileless Malware

Malicious code that resides only in memory (RAM) and leverages legitimate applications or native tools to operate, leaving few or no on-disk artifacts for traditional antivirus to scan.

⚠️ ***Exam Trap:*** *Memory injections são uma forma de ataque fileless: o código vive na memória de processos legítimos, não no disco. Gatilho 'não deixa rastros no disco / roda só em memória' = fileless, exige detecção comportamental (EDR).*

• Privilege Escalation

The attacker objective of gaining higher rights or a trusted execution context by running malicious code inside legitimate processes, bypassing security controls that monitor untrusted applications.

⚠️ ***Exam Trap:*** *DLL injection e process hollowing são TÉCNICAS usadas para escalar privilégio/evadir defesa: rodar no contexto de processo confiável concede acessos extras. Se a questão pede o OBJETIVO → privilege escalation; se pede a TÉCNICA → hollowing ou injection.*

✅ Authentic Practice Questions (10 Real Exam Scenarios)

Scenario-based exam questions. Complete explanations and distractor analysis are provided in the answer key below.

QUESTION 1 · EXAM SCENARIO

A security analyst observes that WINWORD.EXE, which is signed and expected, has caused an unsigned module named update.dll to load inside EXPLORER.EXE. EDR telemetry also shows WriteProcessMemory and CreateRemoteThread calls from WINWORD.EXE to EXPLORER.EXE. Signature-based antivirus does not alert. Which memory injection technique is most likely being used?

A) Process hollowing

B) DLL injection

C) DLL search order hijacking

D) Reflective DLL loading

QUESTION 2 · EXAM SCENARIO

During incident response, an analyst finds that a signed system process was created in a suspended state. Its original memory image was unmapped, malicious code was written into the empty address space, and the process was resumed. The executable on disk still appears legitimate. Which technique does this describe?

A) DLL injection

B) Thread execution hijacking

C) Process hollowing

D) Remote shellcode injection

QUESTION 3 · EXAM SCENARIO

An organization's signature-based antivirus reports no infection on an endpoint. However, EDR shows malicious code executing inside the memory of a trusted, digitally signed application, and no malicious executable is found on disk. Which factor best explains why the antivirus missed the activity?

A) The antivirus engine was disabled by Group Policy.

B) The malware encrypted the entire disk before executing.

C) The trusted application blocked all file system scans.

D) The malicious code runs in a trusted process and may not exist as a file with a recognizable signature.

QUESTION 4 · EXAM SCENARIO

A security team wants to improve detection of DLL injection and process hollowing across Windows endpoints. Which capability is most appropriate for identifying these behaviors?

A) Signature-based antivirus scanning of the file system.

B) EDR monitoring of process memory, API calls, and parent-child process relationships.

C) Host firewall rules that block outbound port 445.

D) Full disk encryption with TPM-backed keys.

QUESTION 5 · EXAM SCENARIO

A low-privileged user process injects a malicious DLL into a Windows service that is running under the SYSTEM account. The injected code begins executing inside that service. What is the most significant security impact?

A) The malicious code can execute with SYSTEM-level privileges.

B) The service's digital signature is automatically revoked.

C) The DLL is copied to the SYSTEM profile for backup.

D) The user's password hash is immediately reset.

QUESTION 6 · EXAM SCENARIO

EDR logs show a process calling OpenProcess on a remote process, then VirtualAllocEx, WriteProcessMemory, and CreateRemoteThread. The bytes written are directly executable code, not a path to a library, and no new executable or DLL is written to disk. Which activity best matches this sequence?

A) Process hollowing

B) DLL search order hijacking

C) Credential dumping from LSASS

D) Remote shellcode injection

QUESTION 7 · EXAM SCENARIO

An EDR alert states that the memory image of a running process does not match the signed executable stored on disk. The process is otherwise using a legitimate name such as svchost.exe. Which technique is the strongest indicator of?

A) DLL injection

B) DLL search order hijacking

C) Process hollowing

D) Keylogging via hardware device

QUESTION 8 · EXAM SCENARIO

A company enforces an application allowlisting policy that permits only approved, signed executables. Malware causes a signed executable to load an unsigned DLL from a user-writable folder, and the DLL executes. Which security control was bypassed?

A) Application allowlisting

B) Network segmentation

C) Multifactor authentication

D) Full disk encryption

QUESTION 9 · EXAM SCENARIO

An analyst confirms that a workstation is running a hollowed process containing malicious code. The workstation is still connected to the corporate network. Which action best supports containment and evidence preservation?

A) Immediately reboot the workstation to clear the malicious memory.

B) Isolate the endpoint from the network and capture volatile memory before terminating the process.

C) Run a full signature-based antivirus scan and then continue using the workstation.

D) Delete all files in the user's temporary folder and close the process.

QUESTION 10 · EXAM SCENARIO

A Windows application is vulnerable to memory injection. Administrators want to make it harder for injected shellcode to execute from data-only memory regions. Which Windows exploit mitigation is most relevant?

A) BitLocker drive encryption

B) Kerberos constrained delegation

C) DNS Security Extensions (DNSSEC)

D) Data Execution Prevention (DEP)

ANSWER KEY & COMPREHENSIVE DISTRACTOR EXPLANATIONS

Question 1: Option B — DLL injection

DLL injection occurs when a legitimate process is forced to load a malicious library. The use of WriteProcessMemory and CreateRemoteThread to target another process, followed by loading update.dll, matches classic DLL injection. Process hollowing would replace the process image, not merely add a module. DLL search order hijacking abuses DLL search paths, not necessarily remote memory writes and thread creation. Reflective DLL loading typically maps a DLL directly from memory without passing a file path to LoadLibrary.

Question 2: Option C — Process hollowing

Process hollowing creates a legitimate process in a suspended state, unmaps its original image, writes malicious code into its memory space, and resumes execution. The process keeps a legitimate appearance while the in-memory code is malicious. DLL injection adds a library to an existing process; thread execution hijacking alters an existing thread; remote shellcode injection inserts code without necessarily unmapping the original image.

Question 3: Option D — The malicious code runs in a trusted process and may not exist as a file with a recognizable signature.

Signature-based antivirus depends on known file or pattern signatures on disk. In memory-injection attacks such as DLL injection and process hollowing, malicious code may reside only in the memory of a trusted, signed process. Therefore, there may be no malicious file with a recognizable signature to detect. The other options do not directly explain this limitation.

Question 4: Option B — EDR monitoring of process memory, API calls, and parent-child process relationships.

EDR is effective against memory injection because it monitors behavior, suspicious API calls (for example, WriteProcessMemory, CreateRemoteThread, NtUnmapViewOfSection), memory integrity, and process relationships. Signature-based antivirus may miss code that exists only in memory. Host firewall and disk encryption do not detect in-memory injection.

Question 5: Option A — The malicious code can execute with SYSTEM-level privileges.

When code is injected into a process running as SYSTEM, the injected code generally inherits that process's token and privileges. This enables privilege escalation because the attacker moves from low privilege to SYSTEM-level execution. The service's digital signature is not automatically revoked, the DLL is not copied to the SYSTEM profile for backup, and the user's password hash is not reset by this mechanism.

Question 6: Option D — Remote shellcode injection

The sequence OpenProcess, VirtualAllocEx, WriteProcessMemory, and CreateRemoteThread is typical of remote shellcode injection: allocate memory in the target process, write shellcode, and create a thread to execute it. Because the written bytes are executable code rather than a library path, and no new executable or DLL is written to disk, it does not match the most direct pattern of process hollowing or DLL loading. LSASS credential dumping would involve accessing LSASS memory, but not necessarily this exact sequence.

Question 7: Option C — Process hollowing

Process hollowing is characterized by a legitimate process whose memory has been replaced with malicious code. As a result, the in-memory image does not match the signed executable on disk. DLL injection adds a module but does not necessarily replace the main image. DLL search order hijacking abuses loading of a wrong DLL, and hardware keylogging does not relate to this memory-image mismatch.

Question 8: Option A — Application allowlisting

Application allowlisting typically authorizes executables but may not control every DLL loaded by those executables. If an allowed and signed binary loads an unsigned DLL from a user-writable location, malicious code can execute without directly launching a new unauthorized executable. Network segmentation, MFA, and full disk encryption are not bypassed by this specific mechanism.

Question 9: Option B — Isolate the endpoint from the network and capture volatile memory before terminating the process.

Volatile memory contains critical evidence such as injected code, loaded modules, and process artifacts. Isolating first contains spread, and capturing memory before terminating the process preserves evidence that would otherwise be lost. Rebooting or terminating immediately destroys memory; a signature scan may not detect memory-only code; deleting temporary files does not remove a running injection.

Question 10: Option D — Data Execution Prevention (DEP)

DEP marks memory regions as non-executable, helping prevent code injected into data areas from executing. BitLocker protects data at rest; Kerberos handles authentication and delegation; DNSSEC protects DNS integrity. None of these directly prevent shellcode execution in memory as DEP does.